

Análisis de ingeniería inversa sobre un sistema *legacy*

Base de datos

12 de junio de 2026

Índice

1. Introducción	2
2. Descripción del problema	2
3. Análisis	2
3.1. Estructura de Entidades y Relaciones	3
4. Diagramas y resultados	5
5. Consultas de extracción de datos	6
6. Conclusiones	8

Índice de figuras

1. Diagrama entidad-relación para el modelo de datos del Sistema Escolar, generado mediante ingeniería inversa.	5
---	---

Índice de cuadros

1. Entidad: departamentos	3
2. Entidad: profesores	3
3. Entidad: cursos	3
4. Entidad: estudiantes	3
5. Entidad: inscripciones	3
6. Entidad: aulas	4
7. Entidad: horarios	4
8. Entidad: notas	4
9. Relaciones y dependencias entre tablas.	4

Lista de códigos

1. Script de inserción de datos iniciales en la base de datos escolar 5

1. Introducción

Se nos ha encomendado realizar ingeniería inversa y, posterior, análisis de un conjunto de archivos que comprenden la definición estructural y lógica de la base de datos de un sistema “Legacy”. Con la finalidad de practicar e integrar todo lo aprendido hasta el momento.

2. Descripción del problema

Una institución educativa necesita modernizar y centralizar su control administrativo y académico. Actualmente requiere un sistema de bases de datos relacional que gestione de manera eficiente la información de sus departamentos, el personal docente, la oferta de cursos, las inscripciones de los estudiantes, así como la asignación de espacios físicos y horarios de clase.

3. Análisis

El dominio de la institución se organiza conceptualmente a través de **departamentos**, los cuales identifican las diferentes áreas de la escuela. Cada departamento agrupa a varios **profesores**, de quienes se guarda su información personal y de contacto.

Cada profesor tiene a su cargo la impartición de distintos **cursos**, los cuales tienen un nombre específico y otorgan una cantidad determinada de créditos académicos. Por otro lado, la escuela registra a sus **estudiantes** con sus datos básicos y fecha de nacimiento. Los estudiantes se matriculan en los cursos mediante **inscripciones**, registrando la fecha exacta del trámite. A través de estas inscripciones, el sistema vincula y registra las **notas** o calificaciones obtenidas por el alumno en dicha materia.

Para la logística de las sesiones, la escuela cuenta con **aulas** que poseen una capacidad máxima definida. Las clases se estructuran a través de **horarios**, los cuales vinculan de manera precisa un curso con un aula específica, detallando el día de la semana en que se imparte, junto con la hora de inicio y fin.

Para la documentación de esta arquitectura de datos, se omitió la escritura manual de los scripts DDL en favor de un enfoque visual y automatizado. Se aplicó un proceso de ingeniería inversa (Reverse Engineering) directamente sobre el script SQL de creación de la base de datos. Este procedimiento permitió a la herramienta de modelado leer e interpretar las tablas, atributos, tipos de datos, llaves primarias y las restricciones de llaves foráneas, garantizando una correspondencia exacta con el sistema analizado.

3.1. Estructura de Entidades y Relaciones

A continuación se detalla la composición de cada una de las entidades del sistema, así como las relaciones establecidas mediante llaves foráneas.

Columna	Tipo de Dato
id	INT (PK)
nombre	VARCHAR(100)

Cuadro 1: Entidad: departamentos

Columna	Tipo de Dato
id	INT (PK)
nombre	VARCHAR(100)
apellido	VARCHAR(100)
email	VARCHAR(100)
id_departamento	INT (FK)

Cuadro 2: Entidad: profesores

Columna	Tipo de Dato
id	INT (PK)
nombre	VARCHAR(100)
creditos	INT
id_profesor	INT (FK)

Cuadro 3: Entidad: cursos

Columna	Tipo de Dato
id	INT (PK)
nombre	VARCHAR(100)
apellido	VARCHAR(100)
email	VARCHAR(100)
fecha_nacimiento	DATE

Cuadro 4: Entidad: estudiantes

Columna	Tipo de Dato
id	INT (PK)
id_estudiante	INT (FK)
id_curso	INT (FK)
fecha_inscripcion	DATE

Cuadro 5: Entidad: inscripciones

Columna	Tipo de Dato
id	INT (PK)
nombre	VARCHAR(50)
capacidad	INT

Cuadro 6: Entidad: aulas

Columna	Tipo de Dato
id	INT (PK)
id_curso	INT (FK)
id_aula	INT (FK)
dia_semana	VARCHAR(15)
hora_inicio	TIME
hora_fin	TIME

Cuadro 7: Entidad: horarios

Columna	Tipo de Dato
id	INT (PK)
id_inscripcion	INT (FK)
nota	DECIMAL(4,2)

Cuadro 8: Entidad: notas

Tabla origen	Columna FK	Tabla destino
profesores	id_departamento	departamentos
cursos	id_profesor	profesores
inscripciones	id_estudiante	estudiantes
inscripciones	id_curso	cursos
horarios	id_curso	cursos
horarios	id_aula	aulas
notas	id_inscripcion	inscripciones

Cuadro 9: Relaciones y dependencias entre tablas.

4. Diagramas y resultados

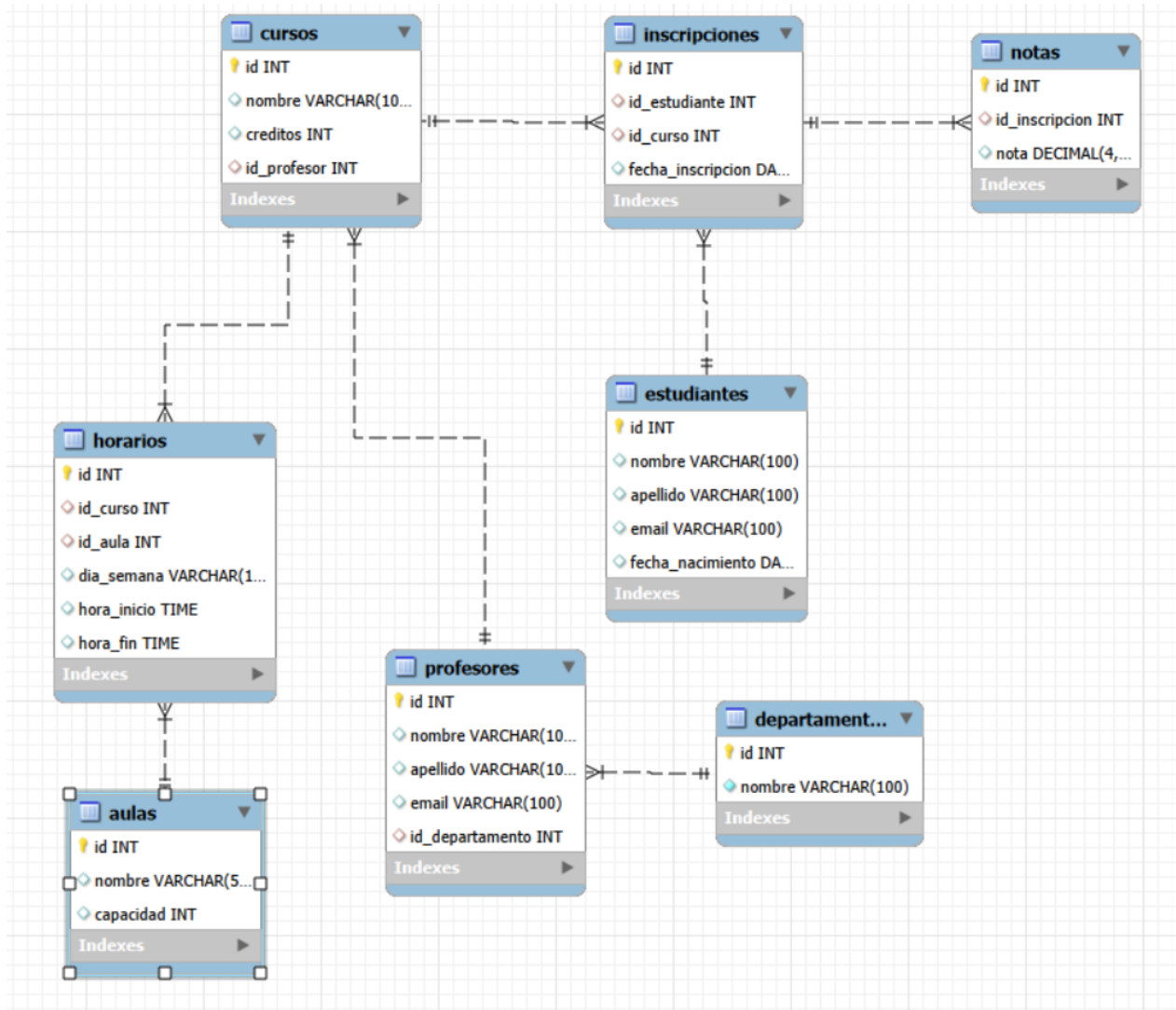


Figura 1: Diagrama entidad-relación para el modelo de datos del Sistema Escolar, generado mediante ingeniería inversa.

Una vez interpretada la estructura física del esquema, se procedió al poblado inicial de las tablas para realizar las pruebas de validación pertinentes. El siguiente script detalla las inserciones de datos maestras y transaccionales ejecutadas para simular el entorno escolar de manera controlada:

```
1 -- Insertar datos en Departamentos
2 INSERT INTO departamentos (nombre) VALUES
3 ('Matemáticas'), ('Ciencias'), ('Humanidades'), ('Ingeniería');
4
5 -- Insertar datos en Profesores
6 INSERT INTO profesores (nombre, apellido, email, id_departamento) VALUES
7 ('Ana', 'Ramírez', 'ana.ramirez@escuela.edu', 1),
8 ('Luis', 'González', 'luis.gonzalez@escuela.edu', 2),
```

```

9 ('Clara', 'López', 'clara.lopez@escuela.edu', 3),
10 ('Pedro', 'Martínez', 'pedro.martinez@escuela.edu', 4);
11
12 -- Insertar datos en Cursos
13 INSERT INTO cursos (nombre, creditos, id_profesor) VALUES
14 ('Álgebra', 5, 1), ('Biología', 4, 2), ('Historia', 3, 3),
15 ('Programación', 6, 4), ('Física', 4, 2), ('Ética', 2, 3);
16
17 -- Insertar datos en Estudiantes
18 INSERT INTO estudiantes (nombre, apellido, email, fecha_nacimiento)
19 VALUES
20 ('Juan', 'Pérez', 'juan.perez@correo.com', '2000-04-15'),
21 ('María', 'Gómez', 'maria.gomez@correo.com', '2001-06-21'),
22 ('Carlos', 'Sánchez', 'carlos.sanchez@correo.com', '1999-12-30'),
23 ('Lucía', 'Díaz', 'lucia.diaz@correo.com', '2002-01-10');
24
25 -- Insertar datos en Inscripciones
26 INSERT INTO inscripciones (id_estudiante, id_curso, fecha_inscripcion)
27 VALUES
28 (1, 1, '2024-01-15'), (1, 2, '2024-01-15'), (2, 1, '2024-01-20'),
29 (2, 3, '2024-01-21'), (3, 4, '2024-01-22'), (4, 5, '2024-01-23');
30
31 -- Insertar datos en Aulas
32 INSERT INTO aulas (nombre, capacidad) VALUES
33 ('Aula 101', 30), ('Aula 102', 25),
34 ('Laboratorio 1', 20), ('Sala Conferencias', 50);
35
36 -- Insertar datos en Horarios
37 INSERT INTO horarios (id_curso, id_aula, dia_semana, hora_inicio, hora_fin)
38 VALUES
39 (1, 1, 'Lunes', '08:00:00', '10:00:00'),
40 (2, 2, 'Martes', '10:00:00', '12:00:00'),
41 (3, 4, 'Miércoles', '13:00:00', '15:00:00'),
42 (4, 3, 'Jueves', '09:00:00', '11:00:00'),
43 (5, 2, 'Viernes', '08:00:00', '10:00:00');
44
45 -- Insertar datos en Notas
46 INSERT INTO notas (id_inscripcion, nota) VALUES
47 (1, 8.5), (2, 9.0), (3, 7.5), (4, 6.8), (5, 9.2), (6, 7.0);

```

Código 1: Script de inserción de datos iniciales en la base de datos escolar

5. Consultas de extracción de datos

Con la base de datos estructurada y poblada, se procedió a responder un listado de preguntas de negocio formulando las correspondientes sentencias SQL de consulta.

1. ¿Cuáles son los estudiantes inscritos en el curso "Álgebra"?

```

1 SELECT e.nombre, e.apellido
2 FROM estudiantes e
3 JOIN inscripciones i ON e.id = i.id_estudiante
4 JOIN cursos c ON i.id_curso = c.id
5 WHERE c.nombre = 'Álgebra';

```

2. ¿Qué cursos imparte el profesor Pedro Martínez?

```

1 SELECT c.nombre AS Nombre_curso,
2 concat(p.nombre, " ", p.apellido) as Nombre_profesor
3 FROM cursos c
4 inner JOIN profesores p ON c.id_profesor = p.id
5 WHERE p.nombre = 'Pedro' AND p.apellido = 'Martínez';

```

3. ¿Cuántos estudiantes están inscritos en cada curso?

```
1 SELECT c.nombre, COUNT(i.id_estudiante) AS total_estudiantes
2 FROM cursos c
3 LEFT JOIN inscripciones i ON c.id = i.id_curso
4 GROUP BY c.id, c.nombre;
```

4. ¿Cuál es el promedio de notas por curso?

```
1 SELECT c.nombre, AVG(n.nota) AS promedio_notas
2 FROM cursos c
3 JOIN inscripciones i ON c.id = i.id_curso
4 JOIN notas n ON i.id = n.id_inscripcion
5 GROUP BY c.id, c.nombre;
```

5. Lista de estudiantes con notas mayores a 8.0

```
1 SELECT e.nombre, e.apellido, n.nota, c.nombre AS curso
2 FROM estudiantes e
3 JOIN inscripciones i ON e.id = i.id_estudiante
4 JOIN notas n ON i.id = n.id_inscripcion
5 JOIN cursos c ON i.id_curso = c.id
6 WHERE n.nota > 8.0;
```

6. ¿Qué cursos tienen más de una inscripción?

```
1 SELECT c.nombre, COUNT(i.id) AS total_inscripciones
2 FROM cursos c
3 JOIN inscripciones i ON c.id = i.id_curso
4 GROUP BY c.id, c.nombre
5 HAVING COUNT(i.id) > 1;
```

7. ¿Qué profesores pertenecen al departamento de 'Ingeniería'?

```
1 SELECT p.nombre, p.apellido
2 FROM profesores p
3 JOIN departamentos d ON p.id_departamento = d.id
4 WHERE d.nombre = 'Ingeniería';
```

8. ¿Cuáles son los cursos programados los lunes?

```
1 SELECT c.nombre, h.hora_inicio, h.hora_fin
2 FROM cursos c
3 JOIN horarios h ON c.id = h.id_curso
4 WHERE h.dia_semana = 'Lunes';
```

9. ¿Qué aulas tienen una capacidad mayor a 25?

```
1 SELECT nombre, capacidad
2 FROM aulas
3 WHERE capacidad > 25;
```

10. Lista de cursos con su aula y horario

```
1 SELECT c.nombre AS curso, a.nombre AS aula, h.dia_semana, h.hora_inicio, h.hora_fin
2 FROM cursos c
3 JOIN horarios h ON c.id = h.id_curso
4 JOIN aulas a ON h.id_aula = a.id;
```

11. ¿Qué estudiantes tiene la nota más alta y en qué cursos?

```
1 SELECT e.nombre, e.apellido, c.nombre AS curso, n.nota
2 FROM estudiantes e
3 JOIN inscripciones i ON e.id = i.id_estudiante
4 JOIN cursos c ON i.id_curso = c.id
5 JOIN notas n ON i.id = n.id_inscripcion
6 WHERE n.nota = (SELECT MAX(nota) FROM notas);
```

12. ¿Cuántos profesores hay por departamento?

```
1 SELECT d.nombre, COUNT(p.id) AS total_profesores
2 FROM departamentos d
3 LEFT JOIN profesores p ON d.id = p.id_departamento
4 GROUP BY d.id, d.nombre;
```

13. ¿Qué cursos no tienen estudiantes inscritos?

```
1 SELECT c.nombre
2 FROM cursos c
3 LEFT JOIN inscripciones i ON c.id = i.id_curso
4 WHERE i.id IS NULL;
```

14. ¿Qué estudiantes están inscritos en más de un curso?

```
1 SELECT e.nombre, e.apellido, COUNT(i.id_curso) AS total_cursos
2 FROM estudiantes e
3 JOIN inscripciones i ON e.id = i.id_estudiante
4 GROUP BY e.id, e.nombre, e.apellido
5 HAVING COUNT(i.id_curso) > 1;
```

15. ¿Cuál es el curso con menor promedio de notas?

```
1 SELECT c.nombre, AVG(n.nota) AS promedio
2 FROM cursos c
3 JOIN inscripciones i ON c.id = i.id_curso
4 JOIN notas n ON i.id = n.id_inscripcion
5 GROUP BY c.id, c.nombre
6 ORDER BY promedio ASC
7 LIMIT 1;
```

6. Conclusiones